

2021 Paper

Question 1

(a) Design a class hierarchy for a University to represent the following data and associated functionality. Include a UML diagram for the class hierarchy. Provide details in Java of all member data and the function prototypes (i.e. function headers) of the member functions.
You need to ensure that you include functions to support the problem in part (b) of this
question.

- The university has a large number of **students**. Each student takes a number of modules each year which must count precisely for 60 credits (30 credits per semester).
- Some students are registered with Disability Services and may be entitled to an extra ten minutes for each hour of an exam. They may also be entitled to a low distraction venue.
- Each **module** has an associated number of credits (which must be a multiple of 5credits). Each module has a specified maximum number of students. Each module has a specified form of evaluation in terms of Coursework (counting for 0%-100%) and an Examination (counting for remainder of the marks). Each module is run during a specific semester. There are two semesters each year.

- **Examinations** must be one of **In-Person** (which are scheduled for a specific venue during a specific time period), **Real-time online** (which are scheduled for a specific time period) or **Take-home exam** (which are scheduled for an extended time period of 12 hours), and these Examinations take place at the end of the semester in which a module is taught.
- Each **Examination venue** has a limited capacity (in terms of the number of students that it can hold at any given time). All students (except those who are entitled to extra time) taking a particular module must be scheduled for the same venue at the same time.
- Any venue which takes less than 30 students can be used as a **low distraction venue**.
- Examinations must be scheduled in one (or more) of the three **Examination slots** (9:00-13:00, 14:00-17:00 and 18:00-21:00). Take-home exams are run across all three slots (i.e. from 9:00 to 21:00).
- **Duration**: In-person and real-time online Examinations can be 1 hour, 2 hours, 3 hours or 4 hours long. Take-home Examinations are 12 hours long. Only Examinations of the same duration may take place in an Examination slot in a specific venue.
- The **Examination window** is the period of time (i.e. the days) during which all exams are to be scheduled which is specified by a start date and an end date. No exams are held on Sundays.
- A student may only be scheduled for a single Examination at any given time. Ideally a student will not be asked to sit more than one Examination per day. If they do have to sit more than one Examination per day then the Examinations must not occur in different venues.

(b) Using the class hierarchy you have developed in part (a), develop an algorithm (in pseudo-code) and code it in Java to schedule exams for all modules and all students for a particular semester. Critically analyse your solution and discuss when/if it will fail, how fair it is to students in terms of each student having their exams spread evenly over the Examination window and how efficient it is in terms of using the available Examination venues and time slots.

Question 2

Given an unsorted array `nums` of unique integers of size N, and an integer `target`, we want to find an efficient algorithm that returns the number of index triplets `i, j, k` with `0 ≤ i < j < k < N`, such that

`nums[i] + nums[j] + nums[k] ≥ target`

To achieve this complete the following tasks:

(a) Consider the following example sorted array:

-20, 0, 5 10, 20, 30, 40

Which pairs i, j of indices satisfy the inequality `nums[i] + nums[j] ≥ 20` ? When we discover the pair of indices (n, m) that satisfies the inequality, which other pairs of indices can we immediately say will satisfy the inequality without looking at the numbers stored in those indices?

(b) Implement the following method in Java which, given a sorted array `nums` of integers of size `N`, and integers `startIndex` and `target`, returns the number of index pairs i, j with startIndex ≤ i < j < N, such that

`nums[i] + nums[j] ≥ target`

The running time performance of your implementation under the asymptotic worst case analysis will be marked.

```
int twoSumGeq(int[] nums, int startIndex, int target)
```

(c) Now implement the following method that solves the original problem of the question. It can also call the method `twoSumGeq`.

```
int threeSumGeq(int[] nums, int target)
```

(d) Write an analysis of the asymptotic worst case running time performance of `twoSumGeq` and `threeSumGeq`.

Your implementation above can assume methods implementing standard algorithms, provided that you name the algorithms and state their worst case performance.